Bellcorp Studio – Assignment 09

Order Tracking System – Approach

I am approaching this problem by first understanding what can go wrong in the order flow and then choosing the technologies based on that problem. I do not want to add unnecessary features just because the stack has many technologies. The main thing I need to get right is that an order should always remain in a valid state, even when many updates come at the same time.

1. Understanding the Problem

The system has a simple order flow:

Placed → Packed → Shipped → Delivered

An order starts as Placed. It can then move to Packed, then Shipped and finally Delivered. A customer should be able to check the current status and its history. The assignment also asks for creating orders and updating their status.

At first this looks like a simple CRUD application. But the important part is the concurrency case. A large batch of orders can be updated within a few seconds, and some of those requests can target the same order almost at the same time. So I cannot simply update the status whenever a request comes in. I need to check the current state and make sure the transition is valid.

For example, if an order is currently Packed, one request may want to move it to Shipped. Another request arriving almost at the same time may also try to update that same order. The database needs to decide the correct order of processing rather than allowing two requests to work with the same old state.

2. My Approach to the Problem

My first step is to define the valid state transitions before writing the APIs:

PLACED → PACKED → SHIPPED → DELIVERED

I will keep this rule in the backend. I do not want the frontend to decide whether a transition is allowed because requests can be sent directly to the API.

- Create the order with status Placed.
- Allow only the next valid status transition.
- Store the current status separately so it can be read quickly.
- Store the status history so the complete journey of an order can be viewed.
- Protect concurrent updates with a database transaction and row-level locking.
- Use Redis only where it actually improves performance or protects the API.
- Keep the UI simple and spend the main effort on correctness, scalability and error handling.

This gives me a smaller system that I can actually explain instead of making a large system with features that are not required.

3. Choosing the Technologies

The required stack is React.js, Node.js with Express.js, PostgreSQL, MongoDB and Redis. I am giving each one a specific responsibility.

I would use React with TypeScript for the frontend. The UI only needs to show the order, current status, history and the required actions. I would not spend too much time on UI styling because the assignment is mainly evaluating system design and implementation quality.

Node.js and Express.js will handle the REST APIs and the business logic. The backend will validate the request, authenticate the user, check the order and perform the status transition.

For the main order data I would use PostgreSQL. Orders are transactional data and the assignment specifically expects PostgreSQL to be used for the transactional core. It also gives me transactions and row-level locking, which is useful for the concurrency problem.

For MongoDB, I would keep activity/audit type information. This data is more flexible and does not need to control the actual state of the order. For example, a status update event can be stored with the order ID, old status, new status, actor and timestamp.

Redis will be used as an in-memory layer. The main use I would start with is caching frequently requested order information. I can also use it for rate limiting a write endpoint. I do not want Redis to become the source of truth for order status.

4. Basic System Design

```
React + TypeScript
      |
      v
Node.js + Express API
      |
      +------ Redis
      |       - cache
      |       - rate limiting
      |
      +------ PostgreSQL
      |       - users
      |       - orders
      |       - status history
      |       - transactions
      |
      +------ MongoDB
              - activity / audit events
```

The important point in this design is that PostgreSQL is the source of truth. Redis is only an optimization. MongoDB contains supporting event information. If Redis is unavailable, I should still be able to process an order using PostgreSQL.

5. Database Design

The PostgreSQL part can be kept small for this assignment.

```
users
- id
- email
```

- password_hash
- created_at

orders
- id
- user_id
- status
- created_at
- updated_at

order_status_history
- id
- order_id
- previous_status
- new_status
- changed_by
- changed_at

The orders table keeps the current state. The history table keeps the transitions. This means I do not have to calculate the current status by reading the complete history every time.

I would add indexes to the columns that are commonly used for searching, such as order_id and user_id. If the history becomes large, the history endpoint should also use pagination instead of returning every record without a limit.

For MongoDB, an event can look conceptually like this:

```
{
 orderId: 4021,
 event: "STATUS_UPDATED",
 from: "Packed",
 to: "Shipped",
 actor: "courier-17",
 timestamp: "..."
}
```

6. API Design

I would keep the API small because the assignment does not require a large number of features.

```
POST   /api/auth/login
POST   /api/orders
GET    /api/orders/:id
GET    /api/orders/:id/history
PATCH  /api/orders/:id/status
```

For the status update, the request would contain the requested next status. The backend would first authenticate the user and validate the input. It would then check the current order state inside the transaction before accepting the change.

The API should return normal HTTP status codes instead of returning successful responses for failed operations. For example, 201 for successful order creation, 200 for successful reads/updates, 400 for invalid input or invalid transitions, 401 for authentication problems, 404 when the order does not exist, 409 where a genuine conflict needs to be reported, and 503 for temporary service/database failures.

7. Concurrency

This is the part I would give the most attention to because it is the main problem described in the assignment.

Suppose Order 4021 is currently Packed. Two requests arrive almost together. Request A wants Packed → Shipped. Request B also works with Order 4021. If both requests first read Packed and then update independently, they could make decisions using stale information.

My solution is to use a PostgreSQL transaction and lock the particular order row while its status is being changed.

```
BEGIN TRANSACTION

SELECT order FOR UPDATE

check current status
validate requested transition
update orders
insert status history

COMMIT
```

The important part is that the lock is only on the order being changed. If Order 4021 is being updated, another request for Order 4022 does not have to wait for Order 4021. This is better than putting one global lock around all orders.

The status update and the history insert should be in the same transaction. If something fails, both can be rolled back. That way I do not end up with an order saying Shipped while its history does not contain the corresponding transition.

I would also make sure that the transition is checked again after the row is locked. The value read before obtaining the lock should not be trusted for the final decision.

8. Redis and Performance

The current order status is likely to be read much more often than it is changed. Therefore I can cache a read such as GET /api/orders/:id.

```
GET order:4021

Cache hit  -> return cached result
Cache miss -> PostgreSQL -> put result in Redis
```

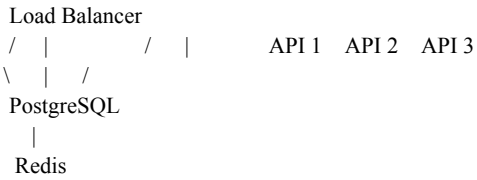
When an order status is successfully changed, I would invalidate the cache key for that order. I would not update the Redis value before the PostgreSQL transaction succeeds. This avoids serving a cached state that was never actually committed.

Redis can also be used for rate limiting a write endpoint such as the status-update endpoint. This provides a second real use without making Redis responsible for the actual order state.

For database performance, I would use indexes, pagination, connection pooling and avoid N+1 queries. I would also avoid returning unlimited history records.

9. Scalability

I am designing the API so that the Node.js layer can remain stateless. If traffic increases, multiple Node.js instances can run behind a load balancer.



For larger traffic, PostgreSQL connection pooling and read replicas can help. Redis can reduce repeated reads. Indexes become more important as the number of orders grows, and history/list endpoints should always be paginated.

The concurrency design also scales better than a single global lock. Different orders can be processed at the same time because each order has its own row-level lock. Only requests competing for the same order need to wait.

10. Error Handling and Edge Cases

I want errors to be handled explicitly rather than letting database errors or stack traces reach the user.

- Invalid request body → 400 Bad Request.
- Invalid status transition such as Placed → Shipped → reject it.
- Backward transition such as Shipped → Packed → reject it.
- Delivered → any other status → reject it because Delivered is the final state.
- Unknown order ID → 404 Not Found.
- Missing or invalid JWT → 401 Unauthorized.
- Unauthorized access to another user's order → reject according to the authorization rule.
- Two requests for the same order → serialize the update using the PostgreSQL row lock and validate the latest state.
- Database failure → return a safe service error instead of a raw database error.
- Redis failure → continue using PostgreSQL where possible because Redis is not the source of truth.
- Failure while updating the order/history → rollback the transaction.
- Large history → use pagination instead of an unbounded query.

One important edge case is repeated status updates. I would not blindly assume that a repeated request is safe. The current state is checked inside the transaction and the API responds according to the transition rule. This keeps the state machine as the authority.

11. Security

- Use JWT authentication for the user flow.
- Hash passwords with bcrypt rather than storing plain text passwords.
- Validate input on the backend.
- Use parameterized database queries or safe ORM/query-builder methods.
- Keep database credentials and JWT secrets in environment variables.
- Add basic rate limiting to a write endpoint.
- Do not expose raw stack traces, database credentials or internal error details.

The frontend is not trusted. Even if the React UI only displays the correct next status, someone can call the API directly, so the backend must enforce the same rules.

12. Folder Structure

I would keep the project structure simple and add files only when there is a real need for them.

```
order-tracking-system/
├── client/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── services/
```

```
│   └── types/
│
├── server/
│   └── src/
│       ├── controllers/
│       ├── routes/
│       ├── services/
│       ├── middleware/
│       ├── models/
│       ├── db/
│       └── utils/
│
├── README.md
├── docker-compose.yml
└── .env.example
```

The main reason for separating controllers, services and database-related code is to keep responsibilities clear. I would avoid creating extra layers or files only to make the structure look bigger.

13. Efficiency and AI-Assisted Development

My development approach would be to design first and then implement only what is required. I would use AI tools selectively for things such as reviewing the architecture, finding edge cases, checking API design, suggesting test scenarios and reviewing code.

I would not ask an AI agent to generate the whole application in one prompt. That can create unnecessary files, assumptions and features that are outside the problem. Instead, I would give it one focused task at a time and review the output before using it.

- First understand the requirement and write the basic flow.
- Use AI to challenge the architecture and point out missing cases.
- Implement the core order flow.
- Use AI for focused code review and test cases.
- Test the concurrency scenario separately.
- Document important prompts and decisions where required.
- Keep the final implementation aligned with the actual assignment.

This also makes the project easier to explain in the next round because I understand why each part exists instead of depending on generated code that I cannot explain.

14. Final Thinking / Trade-offs

My main trade-off is keeping the first version small. The assignment is more concerned with how the system behaves under concurrency and how it is designed than with building a large UI.

I am also deliberately keeping Redis outside the source-of-truth path. This means the application can still rely on PostgreSQL if Redis goes down, although the database may receive more reads. I think this is a better trade-off for correctness.

If the system later becomes much larger, I would consider read replicas, stronger monitoring, asynchronous processing for activity events, and more advanced caching. I would not add those now unless the actual load requires them.

Overall, my design is based on one main idea: keep the order state correct first, then improve performance and scale around it. PostgreSQL handles the transactional state and concurrency, MongoDB handles flexible activity data, Redis handles fast/repeated operations, and the Node.js API keeps the business rules in one place.